File Class in Java

09 July 2024 10:37

The **'File'** Class in Java, part of the **'java.io'** package represents a file or directory path in the file system. It provides a range of methods for creating, deleting and obtaining information about files and directories. However, it does not handle file content directly instead, it is used to manage file and directory attributes and operations.

Key features and methods of the 'File' Class

Here is a detailed explanation of the most important methods and functionalities provided by the File class, along with examples.

Creating File Objects

You can create file objects to represent files and directories. The constructor does not create a file or directory on the disk. It merely creates a file object that represents the path.

Example:

```
package FileHandling;
import java.io.*;

public class FileCreationExample1
{
   public static void main(String args[]){
      //File object representing a file.
      File file = new File("example.txt");

      //File object representing a directory.
      File directory =  new File("Data");

      //File object representing a path to a file within a directory.
      File nestedFile = new File("Data/example.txt");

      System.out.print("\fFile: " + file.getAbsolutePath());
      System.out.print("\nDirectory: " + directory.getAbsolutePath());
      System.out.print("\nNested files: " + nestedFile.getAbsolutePath());
   }
}
```

Output:

```
BlueJ: Terminal Window - JavaSem4
Options
File: D:\Students\Students\SubhalakshmiUEM\Coding\JavaSem4\example.txt
Directory: D:\Students\Students\SubhalakshmiUEM\Coding\JavaSem4\Data
Nested files: D:\Students\Students\SubhalakshmiUEM\Coding\JavaSem4\Data\example.txt
```

Checking File existence

You can check whether a file or a directory exists using the **'exist()'** method.

For example:

```
package FileHandling;
import java.util.*;
import java.io.*;


/**
 * Write a description of class FileExistenceExample here.
 *
```

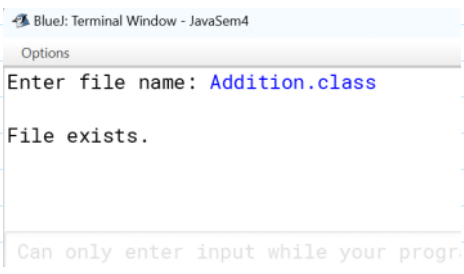
```
 * @author (your name)
 * @version (a version number or a date)
 */
public class FileExistenceExample
{
   public static void main(String args[]){
      Scanner sc = new Scanner(System.in);
      String fileName;
      System.out.print("\fEnter file name: ");
      fileName = sc.nextLine();
      //here we are using relative addressing
      //the below statement is accessing the current directory or folder
      //File file = new File("FileHandling/"+fileName);
      //the belo
      File file = new File("./Polymorphism/"+fileName);
      if(file.exists()){
         System.out.print("\nFile exists.");
      }
      else{
         System.out.print("\nFile does not exists.");
      }
   }
}
```

current folder or directory

This type of addressing is called relative addressing.

./ ← means move out from current working directory.

Output:



Creating and deleting files

You can create a new file using the **'createNewFile()'** method. And delete an existing file using the **'delete()'** method.

For example creating a file

```
package FileHandling;
import java.util.*;
import java.io.*;


/**
 * Write a description of class FileExistenceExample here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class FileExistenceExample
{
   public static void main(String args[]){
      Scanner sc = new Scanner(System.in);
      String fileName;
      System.out.print("\fEnter file name: ");
      fileName = sc.nextLine();
```

```
        //here we are using relative addressing
        //the below statement is accessing the current directory or folder
        //File file = new File("FileHandling/"+fileName);
        //the belo
        File file = new File("./Polymorphism/"+fileName);
        if(file.exists()){
            System.out.print("\nFile exists.");
        }
        else{
            System.out.print("\nFile does not exists.");
        }
    }
}
```

Delete a file:

```
package FileHandling;
import java.util.*;
import java.io.*;


/**
 * Write a description of class DeleteAFile here.
 *
 * @author (yo
ur name)
 * @version (a version number or a date)
 */
public class DeleteAFile
{
    public static void main(String args[]){
        Scanner sc = new Scanner(System.in);
        String fileName;
        System.out.print("\fEnter file name: ");
        fileName = sc.nextLine();
        //here we are using relative addressing
        //the below statement is accessing the current directory or folder
        //File file = new File("FileHandling/"+fileName);
        //the belo
        File file = new File("./Data/"+fileName);
        if(file.delete()){
            System.out.print("\nFile deleted successfully.");
        }
        else{
            System.out.print("\nFile does not exists.");
        }
    }
}
```

Creating Directories

You can create a single directory using the **mkdir()** method and <u>multiple nested directory</u> using the **'mkdirs()'** method.

Example of mkdir() method

```
package FileHandling;
import java.util.*;
import java.io.*;
```

```
/**
 * Write a description of class CreateDirectories here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class CreateDirectories
{
   public static void main(String args[]){
      Scanner sc = new Scanner(System.in);
      String dirName;
      System.out.print("\fEnter your directory name:");
      dirName = sc.next();
      File directory = new File(dirName);
      if(directory.mkdir()){
         System.out.print("\nDirectory created successfully.");
      }
      else{
         System.out.print("\nFailed to create directory or directory already exists.");
      }
   }
}
```

It will create directory or folder, in the current project folder.

Multiple nested directories

```
package FileHandling;
import java.util.*;
import java.io.*;


/**
 * Write a description of class CreateMultileDirectories here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class CreateDirectories
{
   public static void main(String args[]){
      Scanner sc = new Scanner(System.in);
      String dirName;
      System.out.print("\fEnter your directory name:");
      dirName = sc.next();
      File directory = new File(dirName);
      if(directory.mkdirs()){
         System.out.print("\nDirectories created successfully.");
      }
      else{
         System.out.print("\nFailed to create directory or directory already exists.");
      }
   }
}
```

Listing files and directories

You can list all files and directories in the directory using **'list()'** or **'listFiles()'** methods.

```
package FileHandling;
import java.io.*;
import java.util.*;


/**
 * Write a description of class ListFiles here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class ListFiles
{
   public static void main(String args[]){
      String directoryName;
      Scanner sc = new Scanner(System.in);
      System.out.print("\fEnter directory name: ");
      directoryName = sc.next();
      File directory = new File(directoryName);
      if(directory.exists() && directory.isDirectory()){
         //String[] fileList = directory.list();
         File[] fileList = directory.listFiles();
         if(fileList != null){
            //for(String fileName : fileList){
            for(File fileName : fileList){
               System.out.print("\n" + fileName.getName());
               System.out.print("\t" + fileName.lastModified());
            }
         }
         else{
            System.out.print("\nThe directory is empty.");
         }
      }
      else{
         System.out.print("\nDirectory does not exist.");
      }
   }
}
```

Get File Information

You can retrieve various attributes of a file or directory, such as its name, path, size and last modified time.

```
package FileHandling;
import java.io.*;
import java.util.*;
import java.text.*;


/**
 * Write a description of class ListFiles here.
 *
 * @author (your name)
 * @version (a version number or a date)
```

```
 */
public class ListFiles
{
   public static void main(String args[]){
      String directoryName;
      Scanner sc = new Scanner(System.in);
      System.out.print("\fEnter directory name: ");
      directoryName = sc.next();
      File directory = new File(directoryName);
      if(directory.exists() && directory.isDirectory()){
         //String[] fileList = directory.list();
         File[] fileList = directory.listFiles();
         if(fileList != null){
            //for(String fileName : fileList){
            for(File fileName : fileList){
               System.out.print("\n" + fileName.getName());
               System.out.print("\t" + fileName.getAbsolutePath());
               SimpleDateFormat sdf = new SimpleDateFormat("MM/dd/yyyy HH:mm:ss");
               System.out.print("\t"+sdf.format(fileName.lastModified()));
               System.out.print("\t" + fileName.isDirectory());
               System.out.print("\t" + fileName.isFile());
               System.out.print("\t" + fileName.length() + " bytes");
            }
         }
         else{
            System.out.print("\nThe directory is empty.");
         }
      }
      else{
         System.out.print("\nDirectory does not exist.");
      }
   }
}
```

it is a file.

not a directory

Output:

last modified time

Enter directory name: D:\Students\Students\SubhalakshmiUEM\Coding\JavaSem4

File Name

absolute path.

package.bluej D:\Students\Students\SubhalakshmiUEM\Coding\JavaSem4\package.bluej 07/12/2024 11:41:08 false true 1339 bytes ← size of file.

README.TXT D:\Students\Students\SubhalakshmiUEM\Coding\JavaSem4\README.TXT 03/11/2024 23:32:20 false true 480 bytes

InterfacesInJava D:\Students\Students\SubhalakshmiUEM\Coding\JavaSem4\InterfacesInJava 03/11/2024 23:32:40 true false 0 bytes

Polymorphism D:\Students\Students\SubhalakshmiUEM\Coding\JavaSem4\Polymorphism 03/22/2024 23:17:52 true false 0 bytes

doc D:\Students\Students\SubhalakshmiUEM\Coding\JavaSem4\doc 03/22/2024 23:20:34 true false 0 bytes

ThreadsForSubhalakshmi D:\Students\Students\SubhalakshmiUEM\Coding\JavaSem4\ThreadsForSubhalakshmi 04/07/2024 09:21:38 true false 0 bytes

Practice D:\Students\Students\SubhalakshmiUEM\Coding\JavaSem4\Practice 04/21/2024 09:48:30 true false 0 bytes

JavaApplet D:\Students\Students\SubhalakshmiUEM\Coding\JavaSem4\JavaApplet 04/28/2024 10:05:10 true false 0 bytes

ExceptionHandling D:\Students\Students\SubhalakshmiUEM\Coding\JavaSem4\ExceptionHandling 06/12/2024 10:24:46 true false 0 bytes

FileHandling D:\Students\Students\SubhalakshmiUEM\Coding\JavaSem4\FileHandling 06/27/2024 10:12:10 true false 0 bytes

Data D:\Students\Students\SubhalakshmiUEM\Coding\JavaSem4\Data 06/27/2024 11:29:56 true false 0 bytes

Renaming and moving Files

You can rename or move a file using the **'renameTo()'** method.

```
package FileHandling;
import java.io.*;
import java.util.*;


/**
 * Write a description of class RenameMoveFileExample here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class RenameMoveFileExample
{
   public static void main(String args[]){
      String fileName1, fileName2;
      Scanner sc = new Scanner(System.in);
      System.out.print("\fEnter file name to rename: ");
      fileName1 = sc.nextLine();
      System.out.print("Enter new file name: ");
      fileName2 = sc.nextLine();

      File oldFile = new File(fileName1);
      File newFile = new File(fileName2);

      if(oldFile.exists()){
         if(oldFile.renameTo(newFile)){
            System.out.print("\nFile rename/moved successfully.");
         }
         else{
            System.out.print("\nFailed to rename/move file.");
         }
      }
      else{
         System.out.print("\nFile does not exist");
      }
   }
}
```

Output:

```
BlueJ: Terminal Window - JavaSem4
Options
Enter file name to rename: D:\Students\Students\SubhalakshmiUEM\Coding\JavaSem4\Data\worldcup2024.txt
Enter new file name: D:\Students\Students\SubhalakshmiUEM\Coding\JavaSem4\Data\worldcup2024July.txt

File rename/moved successfully.
```

Comprehensive Example

Here is a comprehensive example that demonstrates various file operations using the **'File'** class.

```
package FileHandling;
import java.util.*;
import java.io.*;
```

```java
import java.text.*;

/**
 * Write a description of class FileExample here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class FileExample
{
   public static void main (String args []) {
      String fileName;
      Scanner sc = new Scanner(System.in);
      System.out.print("\fEnter file name: ");
      fileName = sc.next();
      File file = new File(fileName);
      try{
         if(file.createNewFile()){
            System.out.print("\nFile created successfully");
         }

         //Write to the File
         try(BufferedWriter writer = new BufferedWriter(new FileWriter(file))){
            writer.write("Hello, World!");
            writer.newLine();
            writer.write("this is a sample file.");
         }

         //Get file information
         System.out.print("\nFile Name: " + file.getName());
         System.out.print("\nAbsolute Path: " + file.getAbsolutePath());
         System.out.print("\nIs Directory: " + file.isDirectory());
         System.out.print("\nIs File: " + file.isFile());
         System.out.print("\nFile Size: " + file.length());
         SimpleDateFormat sdf = new SimpleDateFormat("MM/dd/yyyy HH:mm:ss");
         System.out.print("\nLast Modified: " + sdf.format(file.lastModified()));


      }
      catch(IOException e){}
   }
}
```

```
Enter file name: D:\Students\Students\SubhalakshmiUEM\Coding\JavaSem4\Data\womensCricketWorldCup2024.txt

File created successfully
File Name: womensCricketWorldCup2024.txt
Absolute Path: D:\Students\Students\SubhalakshmiUEM\Coding\JavaSem4\Data\womensCricketWorldCup2024.txt
Is Directory: false
Is File: true
File Size: 37
Last Modified: 07/18/2024 23:23:00
```